



HTTP

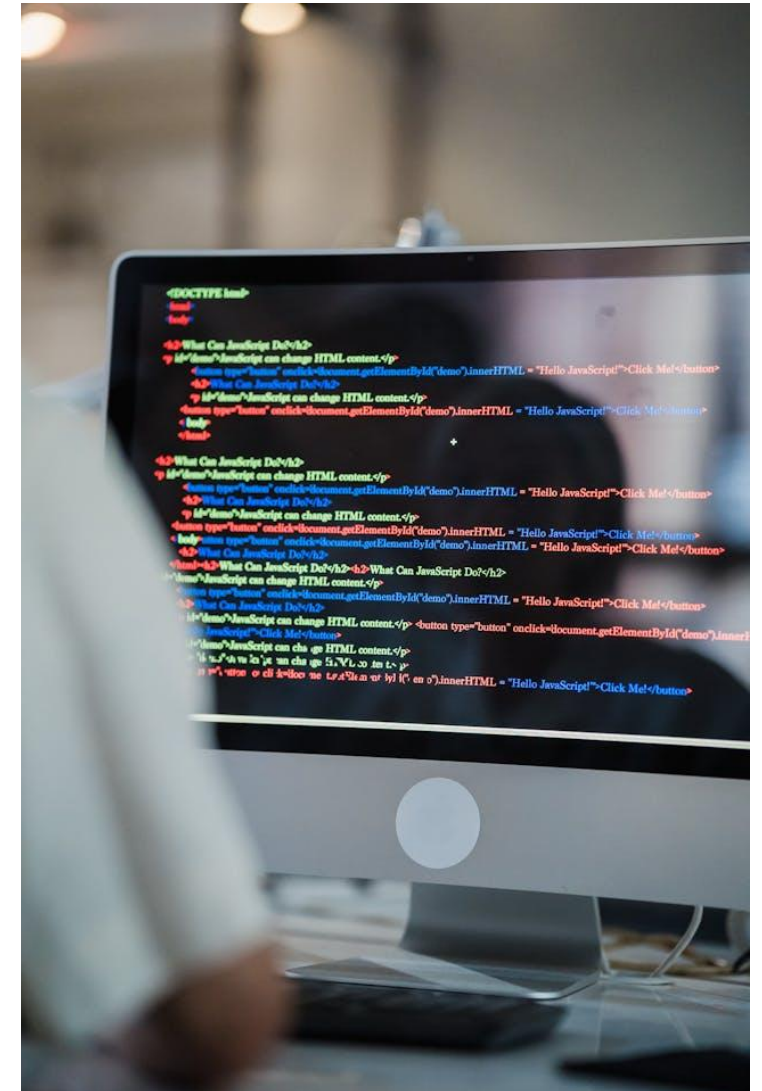
SYSTEM INTEGRATION

Arturo Mora-Rioja
amri@ek.dk

EK

HTTP – HyperText Transfer Protocol

- TCP/IP application level protocol
- Created by Sir Tim Berners-Lee in 1991:
<http://tools.ietf.org/html/rfc2616>
- Runs by default on port 80 over TCP
 - Connection-oriented and reliable
- Contexts of use
 - Web server
 - Intranet server (addresses 10.x.x.x, 172.16.x.x.-172-31.x.x and 192.168.x.x)
 - Local server (default address: localhost or 127.0.0.1)
- It can serve static (HTML) and dynamic pages (CGI, PHP, ASP, Java, JSP, JS...)



URL – Uniform Resource Locator

- Format

- Service://(User:Password@)Server(:Port)(/Path)(/Resource)(?Query=Value)(#Anchor)

- Examples

- <https://ek.dk>
 - <https://ek.dk/english/exchange>
 - https://www.ic3.gov/AnnualReport/Reports/2024_IC3Report.pdf
 - <ftp://speedtest.tele2.net:21>
 - <ftp://demo:password@test.rebex.net:21>
 - https://www.google.com/search?q=ek&rlz=1C1CHBF_enDK1040DK1040&oq=ek&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIHCQEQLhiABDIMCAIQIxgnGIAEGIoFMg0IAxAAGJECGIAEGIoFMg0IBBAAGJECGIAEGIoFMgcIBRAAGIAEMgcIBhAuGIAEMgcIBxAAGIAEMgcICBAAGI8CMgcICRAAGI8C0gEJNDA3NGowajE1qAIMsAIB&sourceid=chrome&ie=UTF-8
 - https://da.wikipedia.org/wiki/Erhvervsakademi_København#Uddannelser_på_fuldtid_på_Erhvervsakademi_København

HTTP Versions

- HTTP/0.9 (1991)
 - No headers or status codes
 - Only supported GET
- HTTP/1.0 (1996)
- HTTP/1.1 (1997)
 - Runs on TCP
 - Verbose headers
 - One request per TCP connection
 - Connection establishment with TCP 3-way handshake
 - Current use: simple servers, legacy systems



© [Kampus Production](#) @ Pexels

HTTP Versions

- HTTP/2 (2015)
 - Runs on TCP
 - Faster than HTTP/1.1:
 - Binary frames
 - Header compression
 - Multiplexing (parallel streams per TCP connection)
 - Request prioritisation
 - Connection establishment with TCP 3-way handshake
 - More secure than HTTP/1.1, as the use of a TLS certificate is mandatory
 - [HTTP/1.1 vs HTTP/2 Comparison](#)
 - Current use: Half of Internet traffic



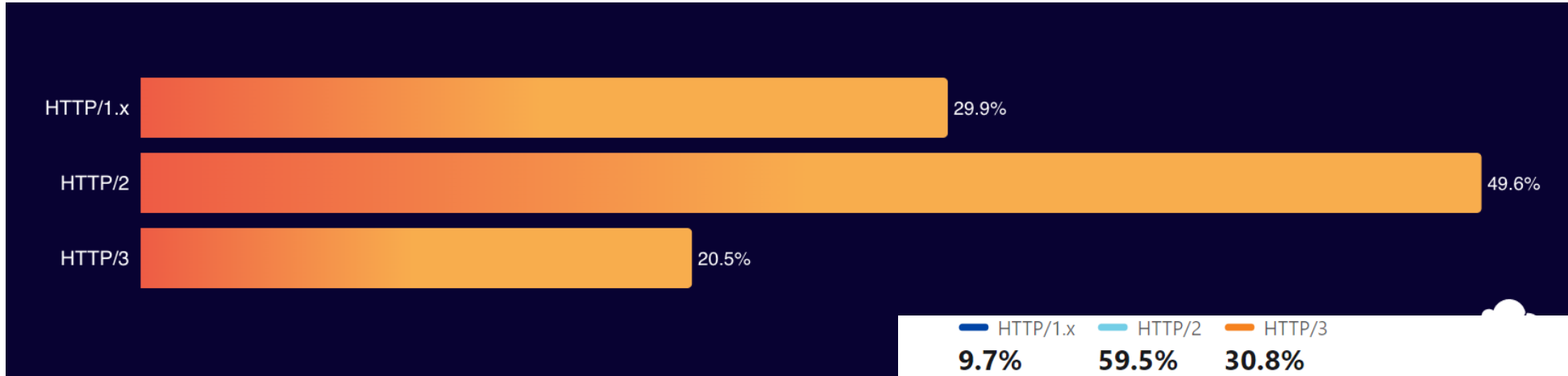
© [Quaritsch Photography](#) @ Unsplash

HTTP Versions

- HTTP/3 (2022)
 - Runs on QUIC (UDP-based)
 - No head-of-line blocking (a blocked item does not prevent following items from delivery)
 - QUIC-based speed advantages
 - Binary frames
 - Header compression
 - Multiplexing at transport level
 - Connection establishment with 0-RTT or 1-RTT QUIC handshake
 - Mandatory use of TLS
 - Good performance on lossy networks, as QUIC handles loss per stream
 - Current use: High-performance sites (CDNs, streaming). Still not universally supported

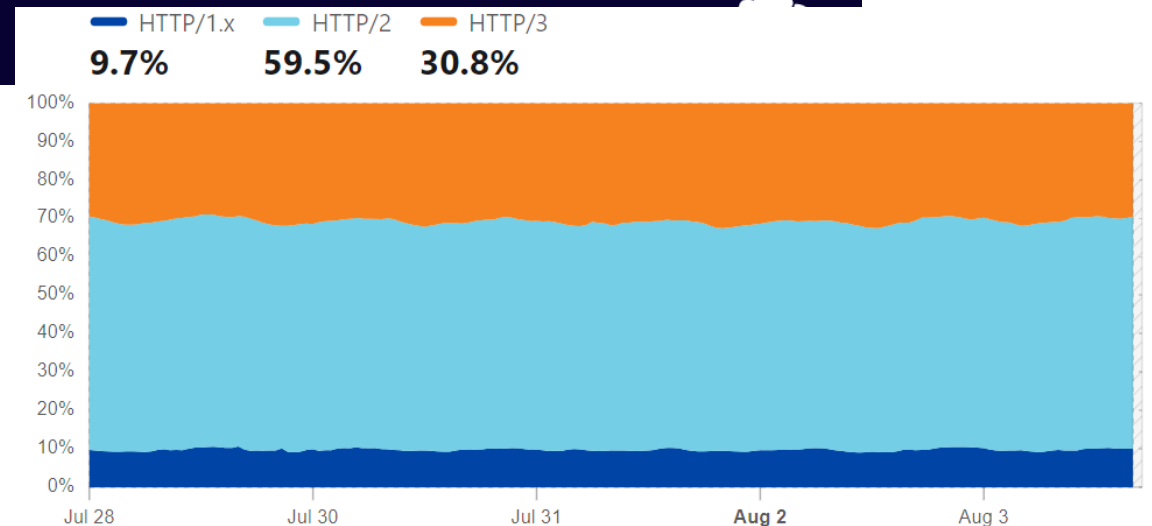
HTTP Versions

- HTTP version support by browser: [HTTP/1.1](#), [HTTP/2](#), [HTTP/3](#)
- Global distribution of traffic by HTTP version in 2024



© 2024 Belson

- Distribution of CloudFlare traffic by HTTP version in 2026



© 2026 [CloudFlare](#)

HTTP Requests and Responses

- HTTP is a request-response protocol

- Request content

- Method
- URL
- Headers
- Body (optional)

GET / HTTP/2

Host: www.ek.dk

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:145.0)
Gecko/20100101 Firefox/145.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br, zstd

Connection: keep-alive

Cookie: ai_user=Il0LowRWGlF/Tmj0yGjEnc|2025-11-05T08:52:26.039Z;

CookieInformationConsent=%7B%22website_uuid%22%3A%228ec17c62-437c-...

Upgrade-Insecure-Requests: 1

Sec-Fetch-Dest: document

Sec-Fetch-Mode: navigate

Sec-Fetch-Site: none

Priority: u=4

TE: trailers

- Response content

- Status
- Headers
- Body (optional)

HTTP/2 200

content-type: text/html; charset=utf-8

date: Thu, 13 Nov 2025 15:16:57 GMT

server: Microsoft-IIS/10.0

content-encoding: gzip

vary: Accept-Encoding

request-context: appId=cid-v1:b01b66ff-a9ac-40e1-a909-5f16da5837ff

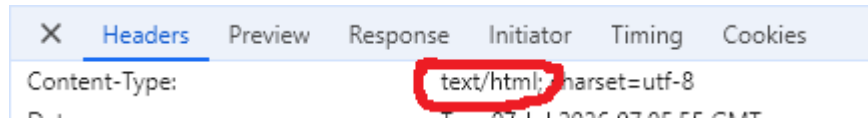
x-powered-by: ASP.NET

X-Firefox-Spdy: h2

<!DOCTYPE html><html lang="da"><head>...

Media Types (MIME Types)

- Multipurpose Internet Mail Extensions
 - Two part identifiers (type = file format/subtype = content format, e.g., application/json)



- Server-set value for the Content-Type HTTP header
 - They inform the user agent about the exact format of the information being interchanged
 - Determined by the [IANA](#) (Internet Assigned Numbers Authority)
 - If set correctly, they aid performance, as the browser can cache files based on type
- Types of types
 - Discrete. Single file or medium: application, audio, example, font, image, model, text, video
 - Multipart. Comprised of multiple component parts, each of which may have their own MIME type: message, multipart

Relevant Media Types

- `application/octet-stream` Default for binary files. Browsers do not execute content
- `text/plain` Default for text files. Browsers display them
- `text/css` , `text/html` If a CSS file is sent as `text/plain`, the browser will not load it as CSS
- `text/javascript` Will be loaded and run as executable JavaScript
- `application/json`
- `application/pdf` Needs proper validation against XSS
- Font types: `font/ttf`, `font/woff`, `font/otf`, `application/vnd.ms-fontobject`
- Image types: `image/avif`, `image/gif`, `image/jpeg`, `image/png`, `image/svg+xml`, `image/webp`
- Audio and video types: `audio/mpeg`, `audio/wav`, `audio/mid`, `video/mp4`, `video/mpeg`, `video/webm`
- [Comprehensive list of MIME types](#)

Form Submission Media Types

- application/x-www-form-urlencoded
 - Default content type for HTML forms sent via POST
 - Inputs are submitted as key-value pairs

```
<form action="http://localhost:8000/" method="post">  
  <label>First name: <input name="txtFirstName" value="Test"></label>  
  <label>Last name: <input name="txtLastName" value="Test"></label>  
  <label>Persons: <input name="txtPersons" type="number" value="2"></label>  
  <label>Phone no: <input name="txtPhoneNo" value="55555555"></label>  
  <label><input type="checkbox" name="chkConsent"> Consent</label>  
  <button>Reserve table</button>  
</form>
```



POST / HTTP/1.1
Host: localhost:8000
Content-Type: application/x-www-form-urlencoded
Content-Length: 80

Notice that the checkbox is
not submitted unless checked
(it would add &chkConsent=on)



txtFirstName=Test&txtLastName=Test&txtPersons=2&txtPhoneNo=55555555

Form Submission Media Types

- multipart/form-data
 - Indicated for mixed data types (e.g., text and binary data)
 - Standard for uploading files
 - High overhead

```
<form action="http://localhost:8000/"
      method="post"
      enctype="multipart/form-data">
  <label>Name:
    <input name="txtName" value="Test">
  </label>
  <label>
    <input type="checkbox"
      name="chkConsent"> Consent
  </label>
  <label>Upload file:
    <input type="picture"
      name="filPic" value="pic.jpg">
  </label>
  <button>Send the picture</button>
</form>
```

```
POST / HTTP/1.1
Host: localhost:8000
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0)
Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data;
  boundary=-----8721656041911415653955004498
Content-Length: 465
-----8721656041911415653955004498
Content-Disposition: form-data; name="txtName"
Test
-----8721656041911415653955004498
Content-Disposition: form-data; name="chkConsent"
on
-----8721656041911415653955004498
Content-Disposition: form-data; name="filPic"; filename="pic.jpg"
Content-Type: image/jpeg
Simple file.
-----8721656041911415653955004498--
```

Adapted from © 2026 MDN and © 2024 Balehithlu

HTTP Messages and Status Codes

Method	Description	Success	Failure
GET	Retrieves a resource. Information is sent as part of the URL	200 OK	404 Not Found
POST	Creates a new resource. Information is sent as part of the request body	201 Created	401 Unauthorised 409 Conflict 404 Not Found
PUT	Updates a resource by replacing it	200 OK	401 Unauthorised 404 Not Found 405 Method Not Allowed
PATCH	Updates part of a resource	200 OK	401 Unauthorised 404 Not Found 405 Method Not Allowed
DELETE	Deletes a resource	200 OK	401 Unauthorised 404 Not Found
OPTIONS	Returns a description of the communication options for a resource	200 OK	
HEAD	Returns the header of a response	200 OK	404 Not Found

- 100-199 Informational
 - 200-299 Successful
 - 300-399 Redirection
 - 400-499 Client error
 - 500-599 Server error
-
- Sometimes POST is used instead of GET due to GET's length limit ($2^{11} = 2048$ bytes)

- [Full list of HTTP status codes](#)

Idempotency

- When calling a request multiple times returns the same results as calling it once, the request is said to be idempotent
 - GET, HEAD, PUT, DELETE, and OPTIONS are idempotent
 - GET /books/45 is idempotent: if called 10 times it will return the same data
 - POST and PATCH are not idempotent
 - POST /books is not idempotent: if called 10 times it will create the resource 10 times
 - Risk of duplicate data when
 - Double clicking a send button
 - Refreshing a page with a form
 - Recovering from a network error

Idempotency

- Solution: using a client-generated idempotency key to track duplicate requests
 - For web clients
 - Disabling the submit button upon submission
 - Weak UI-based idempotency-avoiding solution
 - If the frontend is backend generated, the backend can embed a hidden input
 - For SPAs, the key can be stored in session storage

```
<input type="hidden" name="form_submission_id" value="b3f5b2e8-...">
```

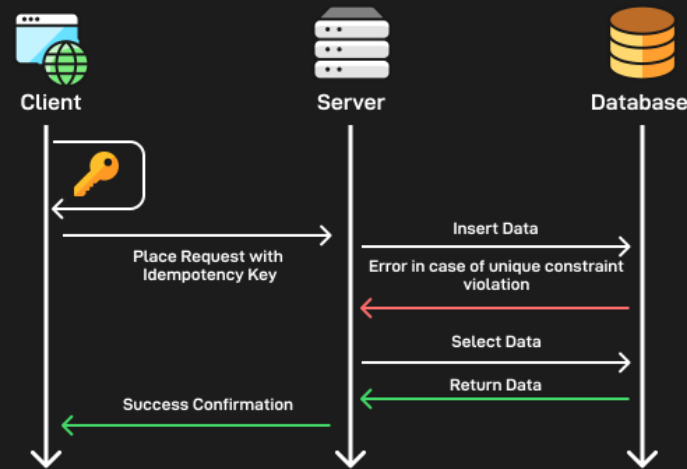
```
const storageKey = 'pending-reservation-idempotency-key';
function getOrCreateIdempotencyKey() {
  let key = sessionStorage.getItem(storageKey);

  if (!key) {
    key = crypto.randomUUID();
    sessionStorage.setItem(storageKey, key);
  }
  return key;
}
```

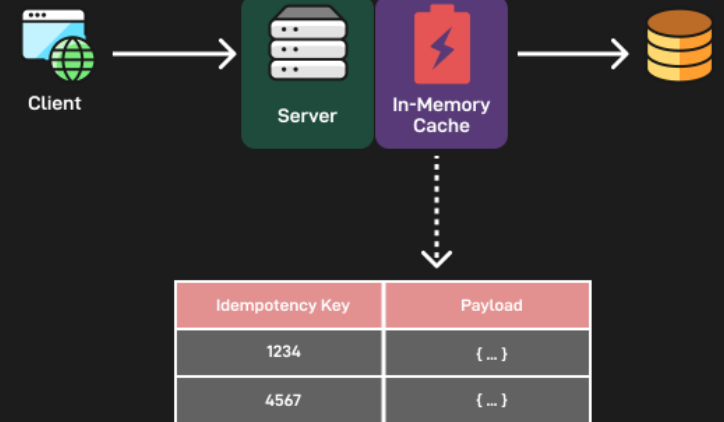
Idempotency

- Ways to implement idempotency for POST and PATCH requests in the backend

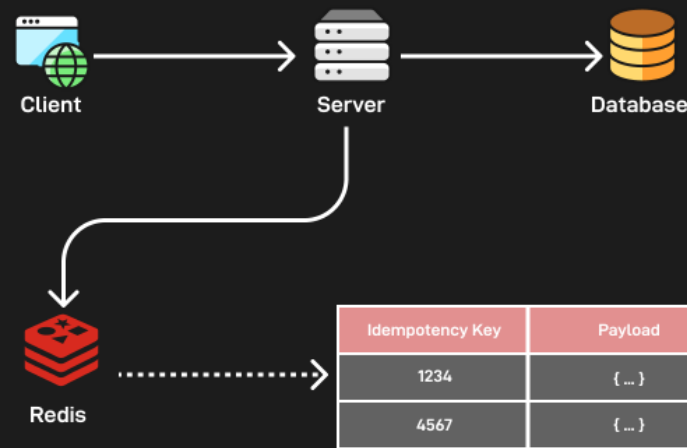
1. Database Unique Constraints



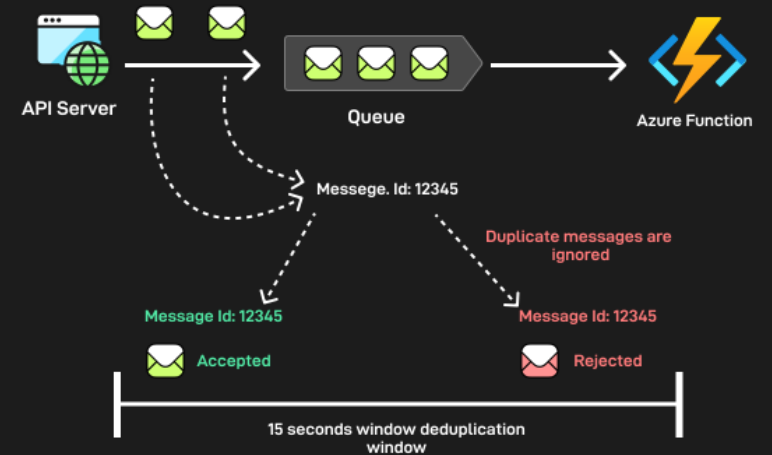
2. In-Memory Tracking



3. Using a Distributed Cache



4. Message Deduplication

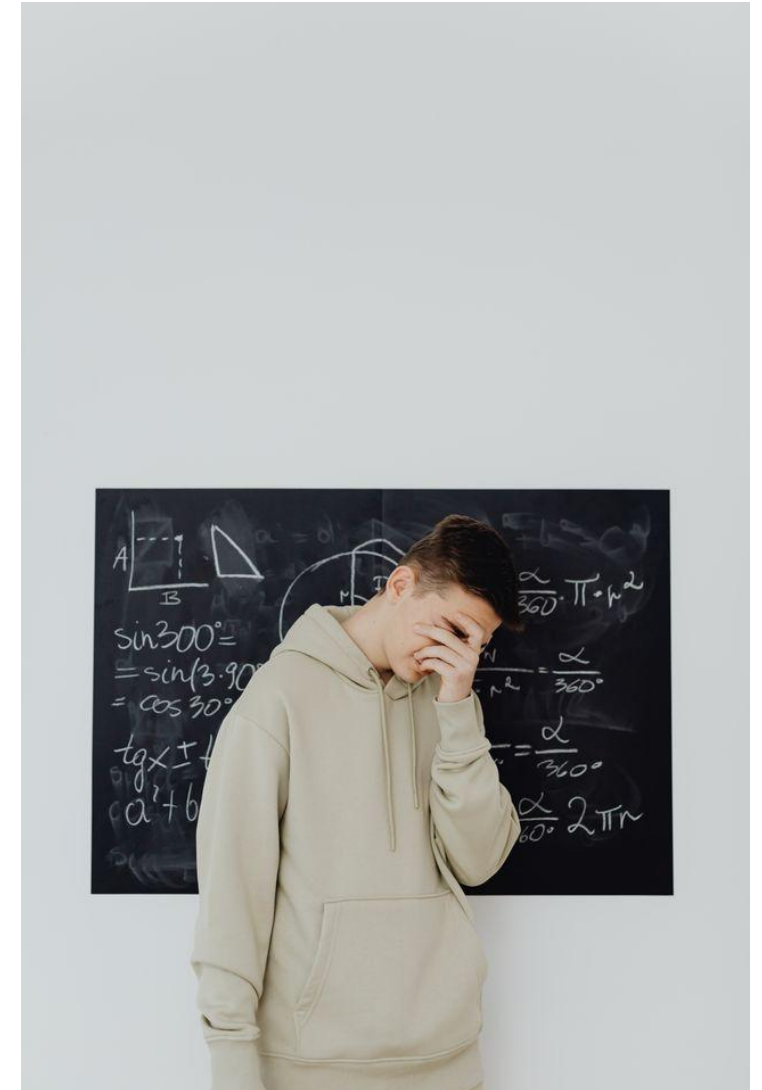


Idempotency Backend Implementation Methods

Method	Pros	Cons
Database unique constraints	Strong consistency via RDB transactions	Requires DB schema support
	Works with multiple server instances	Adds DB writes for idempotency tracking
	Survives server restart	
In-memory tracking	Very easy to implement	Lost upon service restart
	Fast	Does not work with multiple server instances
	No extra infrastructure needed	Needs removal of old keys
		No atomicity: DB write may succeed, key save fail
Using a distributed cache	Scalable: shared across server instances	Requires a key-value store or similar
		No atomicity: DB write may succeed, key write fail
Message deduplication	Ideal for asynchronous systems	Conceptually complex
	Common in event-driven architectures	Usually needs a message broker

GET Request Problems

- Security, as parameter data is part of the URL
- Interoperability, as there are usually limits as to how long the URL can be (down to 8000 characters)
- Traditional solution
 - Using POST requests to retrieve information
 - Parameter data stays in the body of the request
 - The body is not constrained by URL length limits
 - Anti-patterns:
 - Security by obscurity
 - Incorrect use of HTTP verbs



HTTP QUERY Method

- Defined [in June 2026](#)
- Modern solution to GET request issues:
 - Unlike GET, the data is sent in the body of the request
 - Unlike with GET query strings, the payload is not constrained by URL length limits
 - Unlike in usual POST usage, the results are cacheable
 - Unlike POST, it is idempotent
 - Caching occurs at HTTP level
 - The query response itself is cacheable
 - The server may save the query and return a URL for later GET requests, thus avoiding sending the large body again
- 😞 Still not broadly supported

Feature	GET	POST	QUERY
Safe	✓ Yes	✗ No	✓ Yes
Idempotent	✓ Yes	✗ No	✓ Yes
Body allowed	✗ No	✓ Yes	✓ Yes
Cacheable	✓ Yes	⚠ Complex	✓ Yes
Safely replayable	✓ Yes	✗ Risky	✓ Yes
URI for the request	✓ By default	✗ No	✓ Optional

© 2026 Tykok

Serialisation and Deserialisation

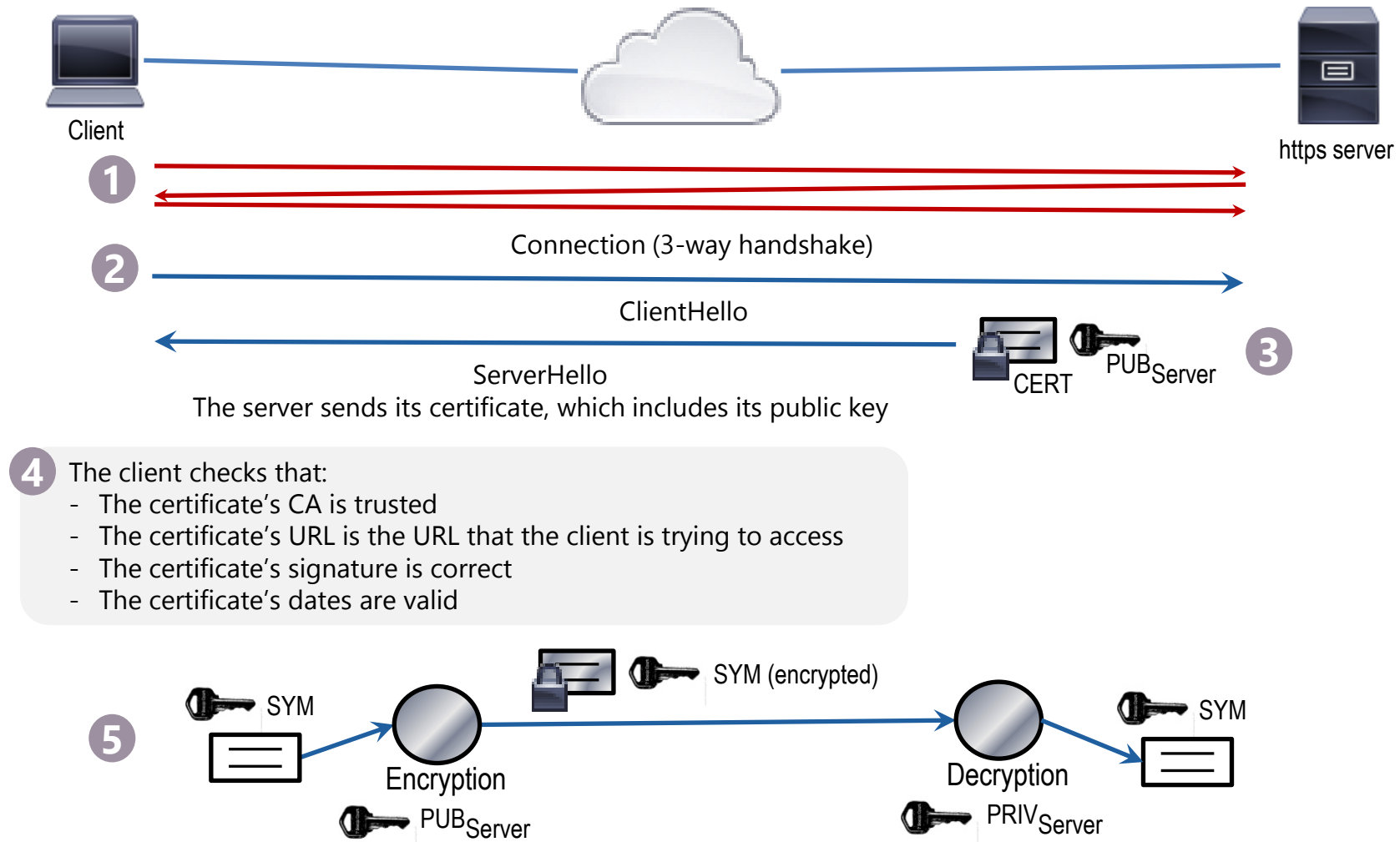
- HTTP payloads are always bytes carrying text
- But what is usually sent/received in web APIs is serialised text (JSON, XML, form-encoded)
- Some frameworks and libraries automatically
 - Serialise text (e.g., into JSON)
 - Deserialise back to text
- Frameworks and libraries that serialise and deserialise automatically
 - Java Spring, .NET, Django REST Framework, PHP Laravel, Axios
- Frameworks and libraries that do not serialise and deserialise automatically
 - Python Flask (use `request.get_json()` and `flask.json jsonify()`)
 - NodeJS (use `response.json()` and `JSON.stringify()`, or the Axios library)

HTTPS – HTTP Secure

- Web servers that serve web sites by HTTPS (default port: 443) have an SSL certificate installed
 - The SSL certificate is validated by a CA (Certificate Authority)
 - Web browsers have a list of trusted CAs
 - When opening a URL, the browser checks that the server's CA is in its list of trusted CAs
- Common CAs: Sectigo, DigiCert, Entrust Datacard, GeoTrust, GlobalSign, GoDaddy, SSL.com
- Self-signed certificates are an alternative to CA-validated certificates
 - They can be used within an organization with several branches, so that third-party trust is not necessary
 - They can be used for internal tests
- Free certificates: Let's Encrypt, CloudFlare, ZeroSSL
 - The domain must be registered in DNS records, though
- [SSL certificate usage](#)

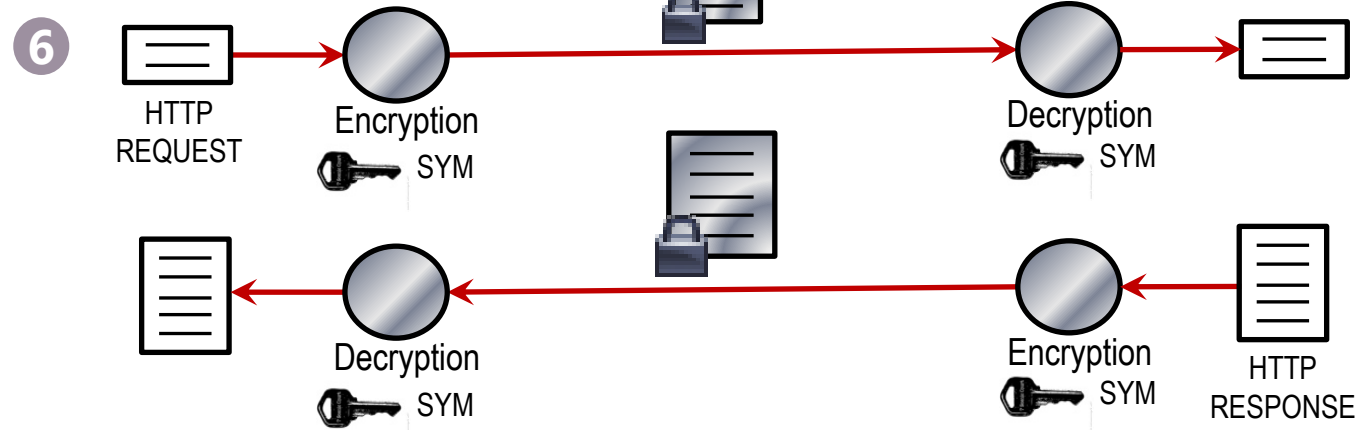


HTTPS Over TCP



The client generates the symmetric key that will be used during the transmission, encrypts it with the server's public key and sends it to the server, which decrypts it with its own private key

HTTPS



All data exchange from this moment on is encrypted with the symmetric key



Disconnection (4-way handshake)

Sources

- **Balehithlu, Ananya.** ["POST Form-Data vs x-www-form-urlencoded: Choosing the Right One"](#). Medium, 2024.
- **Belson, David.** ["Cloudflare 2024 Year in Review"](#). The Cloudflare Blog, 2024.
- **Chinta, Naveena.** ["Designing Idempotent Systems – Beyond Payment APIs"](#). Medium, 2025.
- **MDN.** ["Media types \(MIME types\)"](#). MDN, 2026.
- **markupninja@hachyderm.io.** ["Demistifying MIME Types: Basics, Use Cases, Best Practices"](#). FrontEndDogma.com, 2025.
- **Mora-Rioja, Arturo, and Mercedes Rodríguez Villafáfila.** *Redes locales e Internet: Conceptos y práctica [Local Area Networks and the Internet: Concepts and Practice]*. Educàlia, 2014. ISBN: 978-8-494283-602-8.
- **Soshin, Alexey.** ["Pragmatic System Design"](#). Udemy, 2025.
- **Tebeka, Miki.** ["gRPC in Python"](#). LinkedIn Learning, 2022.
- **Tykok.** ["The new HTTP method : QUERY"](#). Dev.to, 2026.
- Cover picture by [cottonbro studio](#) from [Pexels](#)